

# Mettre en place un DHCP puis DNS (primaire et secondaire) via bind9 sur une Debian 12



## Sommaire

|                                                                                    |          |
|------------------------------------------------------------------------------------|----------|
| <b>Sommaire.....</b>                                                               | <b>1</b> |
| <b>Légende.....</b>                                                                | <b>2</b> |
| <b>Informations additionnelles.....</b>                                            | <b>2</b> |
| <b>Partie 1 - Prérequis.....</b>                                                   | <b>2</b> |
| Sous partie 1 - Création d'une machine.....                                        | 2        |
| Sous partie 2 - Installation de debian 12.5 sur la VM.....                         | 2        |
| Sous partie 3 - Configuration de l'accès SSH.....                                  | 2        |
| Sous partie 4 - Nom de la machine virtuelle serveur.....                           | 3        |
| <b>Partie 2 - Installation des services.....</b>                                   | <b>3</b> |
| Sous partie 1 - Installation du service DHCP.....                                  | 3        |
| Sous partie 2 - Installation du service DNS.....                                   | 3        |
| <b>Partie 3 - Configuration du DHCP.....</b>                                       | <b>3</b> |
| Sous partie 1 - Configuration de la machine serveur afin d'accueillir le DHCP..... | 3        |
| Sous partie 2 - Configuration du DHCP.....                                         | 4        |
| <b>Partie 3 - Configuration du DNS.....</b>                                        | <b>8</b> |
| Sous partie 1 - Configuration du DNS directe et inversé.....                       | 8        |
| Sous partie 2 - Test du DNS primaire (direct+inversé).....                         | 12       |
| Sous partie 3 - Configuration du DNS secondaire.....                               | 13       |
| Sous partie 4 - Test du DNS secondaire.....                                        | 18       |



# Légende

- **Commande** : Le texte affiché de cette couleur correspond à une commande.
  - **Champs** : Le texte affiché de cette couleur correspond à un champ entré durant le processus d'une commande.
  - **Fichier** : Le texte affiché de cette couleur correspond à un nom de fichier ou à un chemin.
- 

## Informations additionnelles

- Os utilisé : Linux - Debian 12.5 (pour les serveurs) / Windows 10 (pour les clients)
  - Matériel utilisé : 2 machine windows 10 (2 clients) / 2 machines debian 12.5 (2 serveurs) / Un switch cisco 2960 / 2 câbles ethernet
  - Pour ce genre de procédure, merci de ne pas copier et coller les commandes. Elles ont subi un formatage et donc ne peuvent pas être fonctionnelles. Merci de les écrire soit-même.
- 

## Partie 1 - Prérequis

### Sous partie 1 - Création d'une machine

- Pour cette procédure nous avons besoin d'une machine. Dans notre cas nous utiliserons une machine virtuelle que nous allons créer via VirtualBox. Pour ce faire, veuillez vous rendre sur [la procédure de création d'une Vm](#).
  - A noter que pour cette procédure, une vm n'est pas nécessaire, l'objectif et seulement d'avoir debian 12.5
- 

### Sous partie 2 - Installation de debian 12.5 sur la VM

- Pour cette procédure nous avons besoin d'installer debian dans sa version 12.5 sur notre VM. Pour ce faire, veuillez vous rendre sur [la procédure d'installation de debian 12.5](#).
- 

### Sous partie 3 - Configuration de l'accès SSH

- Pour cette procédure nous avons besoin d'activer un accès en ssh sur notre machine virtuelle. Pour ce faire, veuillez vous rendre sur [la procédure de configuration d'accès en ssh](#).



---

## Sous partie 4 - Nom de la machine virtuelle serveur

→ Pour cette procédure nous allons donner le nom de 'debian-ap' à la machine.  
Pour se faire il faudra se rendre dans :

- `nano /etc/hostname`

Une fois cela fait, alors ajouter le nom comme sur l'image ci-dessous

```
GNU nano 7.2 /etc/hostname
debian-ap
```

---

## Partie 2 - Installation des services

### Sous partie 1 - Installation du service DHCP

On va installer le service DHCP sur la Vm avec debian 12.5 qui est notre machine serveur.

→ `apt update`  
→ `apt install isc-dhcp-server`  
→ `systemctl status isc-dhcp-server` (pour vérifier que le service dhcp est bien lancé)

---

### Sous partie 2 - Installation du service DNS

On va installer le service DNS sur la Vm avec debian 12.5 qui est notre machine serveur.

→ `apt update`  
→ `apt install bind9`  
→ `systemctl status bind9` (pour vérifier que le service dns est bien lancé)

---

## Partie 3 - Configuration du DHCP

### Sous partie 1 - Configuration de la machine serveur afin d'accueillir le DHCP

Pour fonctionner, notre machine va avoir besoin d'une ip statique que nous allons lui attribuer.



→ `nano /etc/network/interfaces`

→ Nous allons devoir commenter (avec #) la ligne “iface enp0s3 inet dhcp”

→ En dessous de cette ligne, nous allons ajouter “iface enp0s3 inet static”

→ l'adresse ip sera 192.168.10.1

→ Le masque sera 255.255.255.0

→ La passerelle par défaut sera 192.168.10.254

Voici comment doit être le résultat :

```
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug enp0s3
#iface enp0s3 inet dhcp
iface enp0s3 inet static
address 192.168.10.1
netmask 255.255.255.0
gateway 192.168.10.254
```

⚠ Pour que le changement opère, il ne faut surtout pas oublier une étape. ⚠

Nous devons désactiver et réactiver la carte réseau via les commandes suivantes :

→ `ifdown enp0s3`

→ `ifup enp0s3`

---

## Sous partie 2 - Configuration du DHCP

Nous allons commencer par configurer le service DHCP pour qu'il puisse attribuer automatiquement des adresses au sein du réseau.

→ `nano /etc/dhcp/dhcpd.conf`

- 1) Dans “option domain-name” nous allons devoir mettre le nom de notre domaine.  
Dans l'exemple le domaine sera “92i.local”
- 2) Dans “option domain-name-servers” nous devons ajouter l'adresse ip de la machine serveur. Dans l'exemple la vm est la machine serveur, il faut donc utiliser son adresse de façon à ce qu'elle s'utilise soit même. Nous rentrons alors “192.168.10.1”



- 3) Le “default-lease-time” correspond à la durée du bail DHCP par défaut. Il s'agit de la durée pendant laquelle le client peut utiliser l'adresse IP qui lui a été affectée.
- 4) Le “max-lease-time” correspond à la valeur pour le temps maximal du bail DHCP.
- 5) Il faudra décommenter le “authoritative” ligne 21. Cela indique si le serveur DHCP est le serveur officiel du réseau. C'est ce que nous voulons.
- 6) Nous allons maintenant passer aux déclarations des adresses IP. Rendons nous ligne 40. Nous avons un exemple de déclaration en mode commenté, nous allons nous en servir de modèle. Il faudra déclarer :
  - le subnet (adresse de réseau, donc le dernier octet à 8)
  - le netmask (masque de sous réseau)
  - Les adresse à attribuer avec “range [première-ip dernière ip] donc par exemple si nous voulons attribuer sur le réseau, de l'adresse 100 jusqu'à l'adresse 200, nous allons faire : range 192.168.10.100 192.168.10.200;
  - le “option routers” (passerelle)

Le résultat doit être de cette manière :

```
subnet 192.168.10.0 netmask 255.255.255.0 {  
    range 192.168.10.100 192.168.10.200;  
    option routers 192.168.10.254;  
}
```

Le résultat global du fichier doit être comme ceci :



```
GNU nano 7.2 /etc/dhcp/dhcpd.conf
# dhcpd.conf
#
# Sample configuration file for ISC dhcpd
#
# option definitions common to all supported networks...
option domain-name "92i.local";
option domain-name-servers 192.168.10.1;

default-lease-time 11553;
max-lease-time 7200;

# The ddns-updates-style parameter controls whether or not the server will
# attempt to do a DNS update when a lease is confirmed. We default to the
# behavior of the version 2 packages ('none', since DHCP v2 didn't
# have support for DDNS.)
ddns-update-style none;

# If this DHCP server is the official DHCP server for the local
# network, the authoritative directive should be uncommented.
authoritative;

# Use this to send dhcp log messages to a different log file (you also
# have to hack syslog.conf to complete the redirection).
#log-facility local7;

# No service will be given on this subnet, but declaring it helps the
# DHCP server to understand the network topology.

#subnet 10.152.187.0 netmask 255.255.255.0 {
#}

# This is a very basic subnet declaration.

#subnet 10.254.239.0 netmask 255.255.255.224 {
# range 10.254.239.10 10.254.239.20;
# option routers rtr-239-0-1.example.org, rtr-239-0-2.example.org;
#}

subnet 192.168.10.0 netmask 255.255.255.0 {
range 192.168.10.100 192.168.10.200;
option routers 192.168.10.254;
}
```

Nous devons également définir le type d'ip que nous voulons utiliser.

→ `nano /etc/default/isc-dhcp-server`

Ici nous voulons utiliser l'ipv4 donc nous allons ajouter le nom de la carte réseau qui est "enp0s3". Nous allons devoir décommenter en enlevant le "#" devant la ligne qui correspond à l'ipv4

Le resultat doit etre comme ceci :



```
GNU nano 7.2 /etc/default/isc-dhcp-server
# Defaults for isc-dhcp-server (sourced by /etc/init.d/isc-dhcp-server)

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPDv4_CONF=/etc/dhcp/dhcpd.conf
#DHCPDv6_CONF=/etc/dhcp/dhcpd6.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPDv4_PID=/var/run/dhcpd.pid
#DHCPDv6_PID=/var/run/dhcpd6.pid

# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="enp0s3"
#INTERFACESv6=""
```

→ `systemctl restart isc-dhcp-server` (cela va redémarrer le service dhcp afin de prendre les modifications en compte)

→ `systemctl status isc-dhcp-server` (afin de vérifier si le serveur démarre correctement. Si ce

Afin de s'assurer que cela fonctionne on fait faire :

→ `ifdown enp0s3`

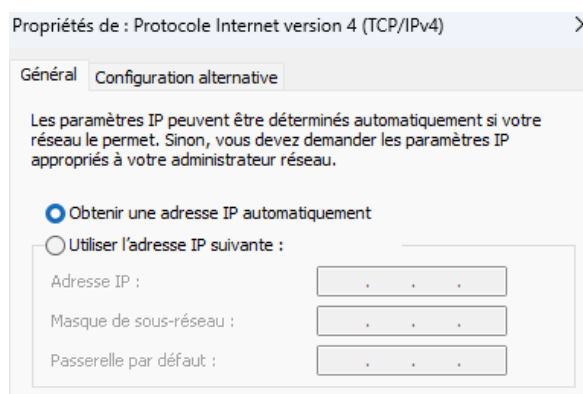
– `ifup enp0s3`

puis également :

n'est pas le cas alors on pourra aller regarder dans le fichier de log "`journalctl -xeu isc-dhcp-server`". Rajouter `.service` si cela ne fonctionne pas

Désormais il faudra effectuer les tests. Notre DHCP devrait fonctionner correctement.

Sur une machine du même réseau, prenons comme exemple un client windows 10, nous allons passer l'obtention de son adresse ip automatiquement et donc géré par le DHCP.



Une fois cela fait, en ouvrant un invite de commande, on fait la commande :

→ `ipconfig*`



Nous devrions trouver comme résultat 192.168.10.101 si nous sommes la première machine. Dans les autres car les 3 premiers octets doivent être similaire et le dernier compris entre 100 et 200 car cela reprend notre déclaration précédente.

[+]

- int fa0/0
- ip helper address [ipserveurdns)

---

## Partie 4 - Configuration du DNS

### Sous partie 1 - Configuration du DNS directe et inversé

Nous allons ajouter le domaine et le nom de serveur pour le DNS :

- `nano /etc/resolv.conf`
- Ajouter "domain 92i.local"
- Ajouter "search 92i.local"
- Ajouter "nameserver 192.168.10.1" (ceci est l'adresse de la machine car c'est serveur, elle va s'utiliser en tant que DNS)

```
domain 92i.local
search 92i.local
nameserver 192.168.10.1
```

Dans le fichier `named.conf.local` on va venir configurer une ou des zones. Une zone DNS est une portion de l'espace de nom du DNS qui est gérée par une organisation ou un administrateur spécifique.

- `nano /etc/bind/named.conf.local`

Configurons le :

- 1) Dans "zone" nous allons ajouter notre nom de domaine. Le "IN" correspond à "Internet" pour indiquer que la zone est destinée à être utilisé sur un réseau TCP/IP
- 2) Dans "type" nous allons configurer le mode de contrôle de zone. Comme nous voulons contrôler la zone, nous allons donc être maître et donc inscrire "master". Il est également possible d'être esclave (slave) mais nous verrons cela par la suite.
- 3) Dans "file" nous allons mettre le nom du fichier de zone direct à l'endroit suivant.  
→ `cd /var/cache/bind/`  
le nom est normé, on a souvent : `db.[domaine]`





Par exemple db.92i.local

Le résultat doit être comme ceci :

```
zone "92i.local" IN {  
type master;  
file "db.92i.local";
```

Pour la zone inversé il faudrait presque faire la même chose :

- 1) Dans "zone" nous allons ajouter les 3 premier octets de notre adresse ip mais inversé donc pour 192.168.10.1 par exemple, on va mettre 10.168.192  
Nous rajouterons après ceci ".in-addr.arpa" IN

in-addr.arpa est l'espace de noms dans .arpa pour les recherches DNS inversées en IPv4

- 2) Nous sommes toujours en "type master"
- 3) Dans "file" nous allons mettre le nom du fichier de zone direct à l'endroit suivant.  
→ `cd /var/cache/bind/`  
le nom est normé, on a souvent : db.[3-premiers-octets]  
Par exemple db.192.168.10

Le résultat final doit être comme ceci :

```
zone "92i.local" IN {  
type master;  
file "db.92i.local";  
notify yes;  
allow-transfer {192.168.10.2;} ;  
};  
  
zone "10.168.192.in-addr.arpa" IN {  
type master;  
file "db.192.168.10";
```

Après avoir défini le fichier de zone direct (db.92i.local) et indirect (db.192.168.10) il faut maintenant les configurer.

Pour la zone direct

→ `nano /var/cache/bind/db.92i.local` (il s'agit du fichier défini précédemment dans `named.conf.local`)



- 1) Le "TTL" (time to leave) correspond aux nombres de secondes qui s'écoulent avant la prise en compte de nouvelles modifications apportées à l'enregistrement. Ici il est de 86400 secondes soit 24h
- 2) Dans le fichier, nous avons plusieurs parties :
  - le domaine suivi d'un point (ici 92i.local.). Il correspond au domaine pour lequel ce fichier de zone est créé
  - "IN" correspond à "Internet" et signifie que la zone est destinée à être utilisée sur un réseau TCP/IP
  - "SOA" indique que cette entrée est l'enregistrement principal de la zone, définissant son autorité.
  - "debian-ap.92i.local." soit [nom-de-la-machine.domaine.] est le serveur dns responsable de la zone
  - "root.92i.local" soit [root.domaine.] correspond à l'adresse e-mail de l'administrateur DNS (le @ est remplacé par un point).
  - "serial" est le numéro de version du fichier de zone qui doit être incrémenté à chaque modification
  - "refresh" correspond au temps après lequel le serveur secondaire doit vérifier s'il y a une mise à jour (ici 7jours / 604800 secondes)
  - "retry" est le délai d'attente avant une nouvelle tentative de mise à jour si la première échoue (ici, 1 jour).
  - "expire" est le temps après lequel un serveur secondaire considère la zone comme invalide si elle n'a pas été mise à jour (28 jours).
  - "negative cache TTL" est le temps de mise en cache des réponses négatives (quand un domaine est introuvable), ici 1 jour.
- 3) Dans la déclaration des serveurs de nom (NS) nous avons plusieurs étapes :
  - 92i.local. est le domaine géré par ces serveurs de noms.
  - "IN" correspond une nouvelle fois à "Internet"
  - "NS" soit "NameServer" définit les serveurs de noms autoritaires pour ce domaine.
- 4) Dans la définition des enregistrements il faudra juste ajouter les ip pour les bonnes machines. Le serveur (debian-ap.92i.local.) à l'ip 192.168.10.1 et le client (poste01.92i.local.) à l'ip 192.168.10.101 qui est l'ip distribué par notre DHCP mis en place précédemment.  
On utilise "A" pour dire que c'est une adresse IPV4



Voici comment le résultat doit être :

```
GNU nano 7.2 /var/cache/bind/db.92i.local
$TTL 86400
92i.local.      IN      SOA      debian-ap.92i.local.  root.92i.local. (
; Serial
604800 ; Refresh
86400  ; Retry
2419200 ; Expire
86400 ) ; Negative Cache TTL

92i.local.      IN      NS       debian-ap.92i.local.

debian-ap.92i.local.  IN      A         192.168.10.1
poste01.92i.local.  IN      A         192.168.10.101
```

Pour la zone indirecte, il n'y a pas grand chose qui va changer. Nous sommes dans le fichier db.192.168.10. Pour rappel dans /var/cache/bind/

- 1) Le premier changement est dans la déclaration NS.
  - in-addr.arpa est utilisé pour la résolution inverse, c'est-à-dire transformer une adresse IP en nom de domaine. La zone concerne le sous-réseau 192.168.10.0/24, représenté par 10.168.192.in-addr.arpa. (les octets sont inversés car c'est une spécificité du DNS inverse).
- 2) Le second changement est dans l'enregistrement.
  - Nous sommes plus en enregistrement de type "A" (pour ipv4) mais en PTR pour mapper une adresse IP vers un nom de domaine.
  - évidemment nous passons la ou les ip avant le "IN" et "PTR" car nous sommes en DNS inversé

Le résultat doit être comme ceci :



```
GNU nano 7.2 /var/cache/bind/db.192.168.10
$TTL      86400
10.168.192.in-addr.arpa.      IN      SOA      debian-ap.92i.local.  root.92i.local. (
    ↵ ; Serial
        604800 ; Refresh
        86400 ; Retry
        2419200 ; Expire
        86400 ) ; Negative Cache TTL
10.168.192.in-addr.arpa.      IN      NS       debian-ap.92i.local.
1.10.168.192.in-addr.arpa.    IN      PTR       debian-ap.92i.local.
101.10.168.192.in-addr.arpa.  IN      PTR       poste01.92i.local.
```

Pour que les changements opèrent, nous allons devoir redémarrer le service bind9 mais afin de limiter nos erreurs nous allons procéder à une vérification.

Pour vérifier les fichiers de configuration et des fichiers de zone (directes et inversés) nous pouvons faire la commande suivante :

→ `named-checkconf -z`

Si il y a des problèmes de syntaxes, nous aurons une erreur avec la ligne de l'erreur et le fichier concerné.

Si aucun problème alors on redémarre le service à l'aide de la commande suivante :

→ `systemctl restart bind9`

→ `systemctl status bind9` (permet de vérifier que le service à correctement redémarré.)

---

## Sous partie 2 - Test du DNS primaire (direct+inversé)

Nous devons obligatoirement vérifier que tout fonctionne correctement à l'aide de certains test.

→ Commençons par le test de la zone direct (nom vers IP) :

1) Sur le serveur on fait la commande suivante :

- `dig -x [nom-de-l'hôte].[domaine]`

Dans notre cas on fera `dig -x poste01.92i.local`

Nous devrions trouver l'adresse IP 192.168.10.101

2) Sur le client on ouvre une invite de commande et on fait :

- `nslookup [nom-de-l'hôte].[domaine]`

Dans notre cas on fera `nslookup poste01.92i.local`

Nous devrions trouver le même résultat



→ Passons ensuite sur le test de la zone inversé (ip vers nom)

1) Sur le serveur on fait la commande suivante :

- `dig -x [adresse-ip]`

Dans notre cas on fera `dig -x 192.168.10.101`

Nous devrions trouver le nom de l'hôte soit `poste01.92i.local`

2) Sur le client on ouvre une invite de commande et on fait :

- `nslookup [adresse-ip]`

Dans notre cas on fera `nslookup 192.168.10.101`

Nous devrions trouver le même résultat

---

## Sous partie 3 - Configuration du DNS secondaire

Pour se faire il va nous falloir une seconde machine sous debian.

Pour sa configuration il nous faut :

- Installer bind9 sur la machine  
→ `apt update`  
→ `apt install bind9`
- Une ip fixe (qui sera 192.168.10.2 dans notre cas)  
On fait → `nano /etc/network/interfaces`

```
GNU nano 7.2 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug enp0s3
iface enp0s3 inet dhcp
iface enp0s3 inet static
address 192.168.10.2
netmask 255.255.255.0
gateway 192.168.10.254
```

Sans oublier `ifdown enp0s3` et `ifup enp0s3`

- [Un accès ssh](#)

Déclaration des zones directes et inversées :

→ `nano /etc/bind/named.conf.local` (ceci est le chemin du fichier `named.conf.local` qui sert pour la déclaration des zones directes et inversées)

C'est ici qu'il faut déclarer les zones DNS que le serveur secondaire (donc cette machine) va récupérer du serveur maître (l'autre premier serveur avec le DHCP)

Pour la zone directe :



- 1) Le nom de la zone ne change pas, on garde le nom de domaine.
- 2) A l'inverse de l'autre serveur, les zones doivent être déclarées comme type "slaves" et non "master" car on veut un dns secondaire  
Par conséquent on doit tout de même indiquer qui sera le maître ("master") en ajoutant son ip (192.168.10.1 dans notre cas)
- 3) Le fichier de zone directe est normé et se nomme slave.[domaine]

Pour la zone inverse :

- 1) Le nom de la zone ne change pas, on garde les 3 derniers octets inversé (car /24) suivi de ".in-addr.arpa"
- 2) On déclare en type "slaves"  
Par conséquent on doit tout de même indiquer également qui sera le maître ("master") en ajoutant son ip (192.168.10.1 dans notre cas)
- 3) Le fichier de zone directe est normé et se nomme slave.[3-premiers-octets]

Voici comment doit être le résultat :

```
GNU nano 7.2 /etc/bind/named.conf.local
//
// Do any local configuration here
//
// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "92i.local" IN {
type slave;
masters { 192.168.10.1; } ;
file "slave.92i.local";
};

zone "10.168.192.in-addr.arpa" IN {
type slave;
masters { 192.168.10.1; } ;
file "slave.192.168.10";
};
```



Nous devons opérer quelques modifications sur le serveur maître (master) afin que le serveur esclave puisse récupérer les zones.

Dans le fichier de configuration de zone (named.conf.local)

- 1) On ajoute dans nos 2 zone "notify yes" qui permet de prévenir immédiatement le serveur slave si une mise à jour d'une zone est effectuée (en lien avec serial)
- 2) On ajoute "allow-transfer" ainsi que son ip (ici 192.168.10.2) pour l'autoriser à copier les informations de zones.

Le resultat doit etre comme ceci :

```
GNU nano 7.2 /etc/bind/named.conf.local
//
// Do any local configuration here
//
// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "92i.local" IN {
type master;
file "db.92i.local";
notify yes;
allow-transfer {192.168.10.2;} ;
};

zone "10.168.192.in-addr.arpa" IN {
type master;
file "db.192.168.10";
notify yes;
allow-transfer {192.168.10.2;} ;
};
```

Dans le fichier de zone direct :

- 1) On doit ajouter un enregistrement NS (NameServer) pour la machine esclave.  
Seul le nom et le domaine de la machine esclave sera différent (on se retrouve avec debian-af.92i.local. dans notre cas



- 2) On doit ajouter un l'enregistrement hôte donc dans notre cas debian-af.92i.local. IN A 192.168.10.2 (on donne juste les informations de la seconde machine serveur)
- 3) Ne pas oublier d'incrémenter manuellement le numéro de série "serial" qui est par défaut sur 1. Cela servira pour le serveur slave.

Le resultat doit etre comme ceci :

```
GNU nano 7.2 /var/cache/bind/db.92i.local
$TTL 86400
92i.local.      IN      SOA      debian-ap.92i.local.  root.92i.local. (
    2 ; Serial
    604800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    86400 ) ; Negative Cache TTL
.
92i.local.      IN      NS       debian-ap.92i.local.
92i.local.      IN      NS       debian-af.92i.local.
.
debian-ap.92i.local. IN      A       192.168.10.1
poste01.92i.local. IN      A       192.168.10.101
debian-af.92i.local. IN      A       192.168.10.2
```

Dans le fichier de zone inversé :

- 1) On doit ajouter un enregistrement NS (NameServer) pour la machine esclave.  
On y met les 3 premiers octets inversés (car masque en /24) et .in-addr.arpa ainsi que "IN", "NS" et le nom. Dans notre cas on aura debian-af.92i.local.
- 2) On doit ajouter un l'enregistrement hôte donc dans notre cas son ip sera noté inversé (donc 2.10.168.192 avec .in-addr.arpa. IN PTR debian-af.92i.local.)
- 3) Ne pas oublier d'incrémenter manuellement le numéro de série "serial" qui est par défaut sur 1. Cela servira pour le serveur slave.

Le resultat doit etre comme ceci :





```
GNU nano 7.2 /var/cache/bind/db.192.168.10
$TTL      86400

10.168.192.in-addr.arpa.      IN      SOA      debian-ap.92i.local.  root.92i.local. (
    2 ; Serial
    604800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    86400 ) ; Negative Cache TTL

10.168.192.in-addr.arpa.      IN      NS       debian-ap.92i.local.
10.168.192.in-addr.arpa.      IN      NS       debian-af.92i.local.

1.10.168.192.in-addr.arpa.    IN      PTR       debian-ap.92i.local.
101.10.168.192.in-addr.arpa.  IN      PTR       poste01.92i.local.
2.10.168.192.in-addr.arpa.    IN      PTR       debian-af.92i.local.
```

Pour que les changements opèrent, nous allons devoir redémarrer le service bind9 mais afin de limiter nos erreurs nous allons procéder à une vérification.

Pour vérifier les fichiers de configuration et des fichiers de zone (directes et inversés) nous pouvons faire la commande suivante :

→ `named-checkconf -z`

Si il y a des problèmes de syntaxes, nous aurons une erreur avec la ligne de l'erreur et le fichier concerné.

Si aucun problème alors on redémarre le service à l'aide de la commande suivante :

→ `systemctl restart bind9`

→ `systemctl status bind9` (permet de vérifier que le service à correctement redémarré.)

Nous devons vérifier sur le serveur esclave qu'il à bien récupéré les fichiers de zone du serveur primaire. On peut s'aider de l'horodatage et des fichiers logs pour confirmer ceci.

→ `ls -l /var/cache/bind`

Nous devons ajouter un nouvel enregistrement sur le serveur primaire et tester la résolution.

La méthode sera la même que précédemment et le nom sera post02 avec l'ip 192.168.10.102

⚠ INCREMENTATION

Voici le résultat de la zone direct :



```

GNU nano 7.2 /var/cache/bind/db.92i.local
$TTL 86400
92i.local.      IN      SOA      debian-ap.92i.local.  root.92i.local. (
    2 ; Serial
    604800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    86400 ) ; Negative Cache TTL
;
92i.local.      IN      NS       debian-ap.92i.local.
92i.local.      IN      NS       debian-af.92i.local.
;
debian-ap.92i.local.  IN      A          192.168.10.1
poste01.92i.local.   IN      A          192.168.10.101
debian-af.92i.local. IN      A          192.168.10.2
poste02.92i.local.   IN      A          192.168.10.102

```

Voici le résultat de la zone inversé :

```

GNU nano 7.2 /var/cache/bind/db.192.168.10
$TTL 86400
10.168.192.in-addr.arpa.      IN      SOA      debian-ap.92i.local.  root.92i.local. (
    2 ; Serial
    604800 ; Refresh
    86400 ; Retry
    2419200 ; Expire
    86400 ) ; Negative Cache TTL
;
10.168.192.in-addr.arpa.      IN      NS       debian-ap.92i.local.
10.168.192.in-addr.arpa.      IN      NS       debian-af.92i.local.
;
1.10.168.192.in-addr.arpa.    IN      PTR      debian-ap.92i.local.
101.10.168.192.in-addr.arpa.  IN      PTR      poste01.92i.local.
2.10.168.192.in-addr.arpa.    IN      PTR      debian-af.92i.local.
102.10.168.192.in-addr.arpa.  IN      PTR      poste02.92i.local.

```

Dans les resolv.conf ([nano /etc/resolv.conf](#)) des 2 serveurs nous avons des modifications a faire

Serveur 1 :

```

GNU nano 7.2 /etc/resolv.conf
domain 92i.local
search 92i.local
nameserver 192.168.10.1
nameserver 192.168.10.2

```



Serveur 2 :

```
GNU nano 7.2 /etc/resolv.conf
domain 92i.local
search 92i.local
nameserver 192.168.10.2
nameserver 192.168.10.1
```

---

## Sous partie 4 - Test du DNS secondaire

Les tests sont les même que précédent avec les commandes :

`dig -x` (côté serveur)

`nslookup` (côté client)

Nous devons obligatoirement vérifier que tout fonctionne correctement à l'aide de certains test.

Ces test sont à effectuer sur la machine client windows 2 (celle qui est aussi en DHCP IP automatiquement avec l'ip en .102)

→ Commençons par le test de la zone direct (nom vers IP) :

3) Sur le serveur on fait la commande suivante :

- `dig -x [nom-de-l'hôte].[domaine]`

Dans notre cas on fera `dig -x poste01.92i.local`

Nous devrions trouver l'adresse IP 192.168.10.101

4) Sur le client on ouvre une invite de commande et on fait :

- `nslookup [nom-de-l'hôte].[domaine]`

Dans notre cas on fera `nslookup poste01.92i.local`

Nous devrions trouver le même résultat

→ Passons ensuite sur le test de la zone inversé (ip vers nom)

3) Sur le serveur on fait la commande suivante :

- `dig -x [adresse-ip]`

Dans notre cas on fera `dig -x 192.168.10.101`

Nous devrions trouver le nom de l'hôte soit poste01.92i.local

4) Sur le client on ouvre une invite de commande et on fait :

- `nslookup [adresse-ip]`

Dans notre cas on fera `nslookup 192.168.10.101`

Nous devrions trouver le même résultat